~\Desktop\C++学习资料\Untitled-1.cpp

```cpp
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
class WritePriorityRWLock {
    mutable std::mutex mtx;
    std::condition_variable cv;
    int readers = 0;
    bool writer = false;
    int waiting_writers = 0; // 核心：等待写者计数

public:
    void read_lock() {
        std::unique_lock<std::mutex> lock(mtx);// 关键：有写者等待时，新读者阻塞
        cv.wait(lock, [this]{ return !writer && waiting_writers == 0; });
        ++readers;
    }
    void read_unlock() {
        std::lock_guard<std::mutex> lock(mtx);
        if (--readers == 0) cv.notify_one(); // 唤醒写者
    }
    void write_lock() {
        std::unique_lock<std::mutex> lock(mtx);
        ++waiting_writers; // 标记有写者等待
        cv.wait(lock, [this]{ return !writer && readers == 0; });
        --waiting_writers;
        writer = true;
    }
    void write_unlock() {
        std::lock_guard<std::mutex> lock(mtx);
        writer = false;// 优先唤醒写者，否则唤醒读者
        if (waiting_writers > 0) cv.notify_one();
        else cv.notify_all();
    }
};

int main() {
    WritePriorityRWLock rwlock;
    auto reader = [&](int id) {
        rwlock.read_lock();
        std::cout << "Reader " << id << " reading\n";
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        rwlock.read_unlock();
    };
    auto writer = [&](int id) {
        rwlock.write_lock();
        std::cout << "Writer " << id << " WRITING\n";
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        rwlock.write_unlock();
    };
    // 测试写优先：读者先启动，但写者2会插队到读者3之前
```

```
    std::thread r1(reader, 1), r2(reader, 2);
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
    std::thread w1(writer, 1); // 开始等待
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
    std::thread r3(reader, 3); // 被阻塞在w1之后
    r1.join(); r2.join(); w1.join(); r3.join();
    return 0;
}
//————————读者优先————————
class RWLock {
    mutable std::mutex mtx;
    std::condition_variable cv;
    int readers = 0;
    bool writer = false;

public:
    void read_lock() {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [this]{ return !writer; });
        ++readers;
    }

    void read_unlock() {
        std::lock_guard<std::mutex> lock(mtx);
        if (--readers == 0) cv.notify_one();
    }

    void write_lock() {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [this]{ return !writer && readers == 0; });
        writer = true;
    }

    void write_unlock() {
        std::lock_guard<std::mutex> lock(mtx);
        writer = false;
        cv.notify_all();
    }
};
```